Quantitative Systems Engineering

Compiled Study Notes & Logs

Jair Aguilar Peña

May 22, 2026

Contents

Chapter 1

Year 01 → Quarter 01 → Notes

1.1 Daily Study Log: 2026-05-21

Source Path: year_01/quarter_01/notes/2026-05-21_log.tex

1.1.1 Session Stats

- **Date:** 2026-05-21
- **Focus Area:** [e.g., Systems Programming / Linear Algebra / Rust]
- **Time Invested:** [e.g., 1 hour 45 minutes]
- **Habit Checklist:**
- [ ] Reading & Theory Study
- [ ] Hands-on coding/math problem solving
- [ ] Notes documented & compiled to NotebookLM

1.1.2 Theory & Reading Summary

What theoretical concepts did I cover today? Include book name/chapter and core notes.

- **Topic/Resource:** [e.g., CS:APP Chapter 2 - Representing and Manipulating Information]
- **Key Concepts learned:**
-
-

1.1.3 Implementation & Practice

What did I build, compile, profile, or debug today? Show code snippets, command lines, or math solutions.

- **Task/Project:** [e.g., Floating point conversions or Rust memory ownership exercise]
- **Key Code/Output:**

```
// Insert code snippets or terminal logs
```

- **What worked/What failed:**
-

1.1.4 Questions & Next Focus

What was difficult or needs further clarification? What is the goal for tomorrow's study session?

- **Blockers/Doubts:**
-
- **Tomorrow's Goal:**
-

Chapter 2

Year 01 → Quarter 01 → Notes → Systems

High level c -> `program.c` low level C -> any binary file compiled
to compile a low level C - any binary file compiled
to compile a `.c` file into binary we use `gcc`. common compile command.
`gcc -o <bin_name> <c_file>` example `gcc -o hello hello.c`
the `gcc` command its an interface to run linked commands:

1. preprocersor `cpp` (converts `.c` files into a modified source program `.i`) modifies directives like #include <stdio.h> reads the his content and insert that content directly in the `.i` resulting file.
2. A compiler `cc1` turns the text modified program into assembly program Text. Converts the `.i` into a text Assembly code litterallly the byte code represented on the numerial bit representation as Text.
3. An Assembler `as` tool turns assembly text file (`.s`) into a relocable object program (`.o` binary) instead of characters character representation the Assembler stores these bytes in a space memory to write the real Assembly Instructions.
4. A linker `ld` that turn the re-locable program into an executable binary file. i.e `hello` binary file no-extention.

2.1 1.4 Processors Reand And interpret instructions stored memory

Source Path: year_01/quarter_01/notes/systems/chapter1_cssapp.tex

Buses. The electrical connection around a computer where the bytes are carried between components. Buses are desingned to transfer fixed size chinks of bytes known as *words*. Most machines today uses either 4 bytes (32bits) or 8 bytes (64 bits).

I/O devices. Systems connection to the external world. keyobard, mouse, a display , a disk drive, etc.

Main memory. is a temporary storage. Memory is organized as a linear array f bytes. each with its own unique adress (array index) starting at 0.

CPU. central progracessing unit, is the engine that interprets or executes instructions stored in main memory.

- At is core is a word sized storae device or register called *program counter*.

- PC points at some chenie language intruction in main memory.
-

ALU. Aritmetic/Logic unit.

Common Processing Unit instructions:

- load: copu a byte or a word from main memory into a register, overwriting the previous contents of the register.
- Store: Copy a bute ot a word from a register to a location in main moemory, overwriting the previous contents of the register.
- Operate: Copyu the contents of two register to the ALU, perform an arthmetic operation the two words. and store the result in a register, overwriting the previous contents of that register.
- Jump: Extract a word from the instruction itself and copy that word in the program counter (PC), overwriting the previous value of the PC.

Save temporary files of compilation for C programs:

```
gcc -save-temps <input> -o <ouput_bin>
```

2.1.1 Summary

`preprocersor` its the responsible of remove coments and ingest built-in code like stdio.h types and functions.

`compiler` its the responsible of optimize the code by re arraging instructions to result into an assembly code, also checks for errors before this previous step and generate the corresponding AS code for the target Architecture (i.e ARM, x86-64).

`Assembler` Transform the `.s` assembly code file into machine code that its a collections of 1 and 0.

`Linker` this is one of the most powerful stages. Since modern compiler don't write all the code into binaries to don't repeat instruction the linker bind some known utilities directly from the System ROM or Drivers and instead of write a new block of code in the binary the binary writes a instruction to execute the already existing instruction.

Your premise that history is irrelevant to a technical focus is a common one, but it is incorrect, especially for foundational technologies like the C language. The historical context is not trivia; it is the technical rationale for the language's design. The text by Dennis M. Ritchie reveals that C's features are not arbitrary rules but pragmatic solutions to specific hardware limitations and software problems of its time. Understanding this history provides a deeper comprehension of why the language works the way it does, which is essential for mastering it.

2.2 C as a Product of Evolution, Not Grand Design

Source Path: year_01/quarter_01/notes/systems/history_of_C_lang.tex

The provided text demonstrates that C was not designed in a vacuum. It evolved directly from the typeless language B, which itself was a stripped-down version of BCPL, created to function on a DEC PDP-7 computer with only 8K of memory. C's core technical characteristics are a direct consequence of this evolutionary path, shaped by three main forces:

- Hardware Constraints: The transition from the word-addressed PDP-7 to the byte-addressable PDP-11 was the primary catalyst for C's most significant feature: its type system. Types were not initially added for safety, but for efficiency and direct hardware mapping.
- Pragmatic Problem-Solving: C was built to write the Unix operating system. Every feature was a tool with a purpose. Features were added, like the ++ and – operators, because they produced more efficient machine code. Features were modified, like the && and || operators, to resolve ambiguity found in its predecessor, B.
- Path Dependence: C retains "fossils" from its ancestors. Decisions made to ease the transition from B, or due to the limitations of early compilers, have left permanent marks on C's syntax and semantics.

Chapter 3

Year 01 → Quarter 01 → Planning

3.1 Week 1: Protocol - Foundations of Representation

Source Path: year_01/quarter_01/planning/week1.tex

This is the operational mandate for the first seven days. The objective is singular: establish a rhythm of deep work in the two core pillars of this year—Computer Systems and Mathematics. Distractions are eliminated. The 2-4 hour block is a non-negotiable commitment.

Adherence to the schedule is the only metric for success this week. The work must be done.

Monday: Computer Science - The Language of Memory

- **Objective:** Master hexadecimal notation, the concept of "word size," and the critical distinction between big-endian and little-endian byte ordering.
- **Block (2 Hours):** Read CS:APP, pp. 29-41 (Sección 2.1.1 a 2.1.4).
- **Execution:**

1. Escribe un programa en C que declare una variable `int x = 0x01234567;` y luego imprima sus bytes individuales para determinar el endianness de tu máquina.
2. Convierte a mano los números decimales 15, 16, 255, y 256 a binario y hexadecimal.
3. Explica en tus notas por qué el byte ordering es un problema fundamental en la programación de redes.

Tuesday: Mathematics - The Geometry of Equations

- **Objective:** To visualize linear equations not as abstract algebra, but as geometric objects: lines, planes, and their intersections.
- **Block (2-3 Hours):** Watch MIT 18.06 Linear Algebra, Lecture 1 ("The Geometry of Linear Equations") and Lecture 2 ("Elimination with Matrices").
- **Execution:**

1. Take handwritten notes. The physical act of writing aids memory.
2. For a 2x2 system, draw the row picture (intersecting lines) and the column picture (vector addition). Do this for a system with one solution, no solution, and infinite solutions.
3. Perform Gaussian elimination on a 3x3 system by hand. Show your work.

Wednesday: Computer Science - Bit-Level Manipulation

- **Objective:** Entender cómo se representan las cadenas de caracteres y el código, y dominar las operaciones a nivel de bit en C.
- **Block (2 Hours):** Read CS:APP, pp. 46-56 (Sección 2.1.5 a 2.1.10).
- **Execution:**

1. Escribe una función en C que use máscaras de bits (`&`) y desplazamientos (`>>`) para extraer el byte más significativo de un entero de 32 bits.
2. Implementa la operación XOR (`^`) usando solo las operaciones AND (`&`) y NOT (`~`).
3. Explica la diferencia fundamental entre el desplazamiento lógico a la derecha y el desplazamiento aritmético a la derecha.

Thursday: Mathematics - Elimination and Inverses

- **Objective:** To master the mechanics of matrix operations, the core machinery of linear algebra.
- **Block (2-3 Hours):** Watch MIT 18.06, Lecture 3 ("Multiplication and Inverse Matrices") and Lecture 4 ("Factorization into A = LU").
- **Execution:**

1. Multiply two 3x3 matrices by hand.
2. Find the inverse of a 2x2 matrix using the formula.
3. Take a 3x3 matrix and perform LU decomposition on it by hand.

Friday: The Forge - Armory and First Contact

- **Objective:** To prepare your development environment and make first contact with your primary language. A warrior's weapon must be sharp and ready.
- **Block (2-3 Hours):**
- **Execution:**

1. Install a C compiler (GCC via MinGW, macOS Command Line Tools, or `build-essential` on Linux).
2. Install Rust via `rustup`.
3. Write, compile, and run `hello, world` in C.
4. Write, compile, and run `hello, world` in Rust using `rustc` directly.
5. Create a new project using `cargo new hello_cargo`, build it, and run it. Read the output to understand what Cargo is doing for you.

Saturday: The Forge - Basic Training

- **Objective:** To move from "hello, world" to programs that perform logic. Solidify basic syntax and concepts.
- **Block (3-4 Hours):** Work through Chapter 3 of *The Rust Programming Language* ("Common Programming Concepts").
- **Execution:**

1. For every concept (variables, data types, functions, control flow), write a small, separate program in a `practice` directory to test it.
2. Build the three programs requested in the book:

- Temperature converter.
- Fibonacci number generator.
- "The Twelve Days of Christmas" lyrics generator.

Sunday: Review and Reload

- **Objective:** To assess the week's progress, identify weak points, and prepare for the next operational cycle.
- **Block (2 Hours):**
- **Execution:**

1. Read through every single note you took this week.
2. Create a "Weak Points" list. Write down the 1-3 concepts that you feel least confident about.
3. Create the `week_02.md` file. The first task for next week will be to attack the items on your "Weak Points" list.
4. Outline the new topics for Week 2 in Year1.md

The plan is updated. The mission is unchanged.

3.2 Week 2: Protocol - Integer Representation and Vector Spaces

Source Path: year_01/quarter_01/planning/week2.tex

The baseline is established. This week, we descend to the next layer of abstraction. You will master the machine's representation of signed and unsigned integers and how mathematicians conceptualize the spaces that vectors inhabit. These topics are dense.

Monday: Computer Science - The World of Integers

- **Objective:** Master the representation of unsigned and signed integers, focusing on the logic of two's complement.
- **Block (2 Hours):** Read CS:APP, pp. 56-69 (Sección 2.2 a 2.2.4).
- **Execution:**

1. For a 4-bit integer, write down all possible values in a table showing their binary, unsigned decimal, and two's complement decimal representations.
2. Calculate by hand the two's complement of -5 and -1 for an 8-bit integer.
3. Explain in your notes why the range of a two's complement number is asymmetric (e.g., -128 to 127 for 8 bits).

Tuesday: Mathematics - The Structure of Vector Spaces

- **Objective:** Move beyond simple vectors to understand the properties of the spaces they inhabit.
- **Block (2-3 Hours):** Watch MIT 18.06, Lecture 5 ("Transposes, Permutations, Vector Spaces") and Lecture 6 ("Column Space and Nullspace").
- **Execution:**

1. Take handwritten notes. Pay close attention to the formal definition of a vector space and a subspace.
2. For a given 3x3 matrix, determine if its column space is a line, a plane, or all of R^3.
3. Find the nullspace for a simple 2x3 matrix. Explain geometrically what the nullspace represents.

Wednesday: Computer Science - The Dangers of Conversion

- **Objective:** Understand the implicit conversions between signed and unsigned integers in C and the subtle bugs they can introduce.
- **Block (2 Hours):** Read CS:APP, pp. 69-79 (Sección 2.2.5 a 2.2.8).
- **Execution:**

1. Write a C program that demonstrates that `(unsigned int)-1 > 0` evaluates to true and explain precisely why this happens at the bit level.
2. Show how an 8-bit integer with a value of -10 is expanded to a 16-bit integer (sign extension).
3. Describe a scenario where the truncation of a number (e.g., casting a `long` to an `int`) could lead to an exploitable security vulnerability.

Thursday: Mathematics - Reinforcement: The Four Subspaces

- **Objective:** Solidify your understanding of the column space and nullspace, which are fundamental to all of linear algebra.
- **Block (2-3 Hours):** Re-watch Lecture 6. Dedicate the entire session to solving problems from the associated problem set.
- **Execution:**

1. For at least three different matrices, find a basis for their column space and their nullspace.
2. Create a matrix `A` where the vector `(1, 2, 3)` is in its column space.
3. Create a matrix `B` where the vector `(1, 1, 1)` is in its nullspace. Explain your reasoning.

Friday: The Forge - The Soul of Rust: Ownership

- **Objective:** To understand Rust's core principle of ownership. This is the single most important concept to master.
- **Block (2-4 Hours):** Read and work through Chapter 4 of *The Rust Programming Language* ("Understanding Ownership").
- **Execution:**

1. Read the chapter without distraction. Type out every single code example and run it.
2. Draw diagrams showing how ownership is transferred when a variable is passed to a function.
3. Create examples that show the difference between a `move` and a `clone`.

Saturday: The Forge - The Borrow Checker is Your Mentor

- **Objective:** To develop an intuitive feel for the borrow checker by intentionally provoking errors and understanding them.
- **Block (3-4 Hours):** Continue practicing concepts from Chapter 4 of the Rust book.
- **Execution:**

1. Write a program that tries to use a value after it has been moved. Read the compiler error carefully. Understand *why* it failed.
2. Write a program that tries to have two mutable references to the same data in the same scope. Analyze the error.
3. Write a function that calculates the length of a string without taking ownership of it (using a reference).

Sunday: Review and Reload

- **Objective:** Consolidate the week's knowledge, identify areas of weakness, and prepare the next week's operational plan.
- **Block (2 Hours):**
- **Execution:**

1. Review all notes from the week. Pay special attention to two's complement, nullspace, and Rust's ownership rules.
2. Update your "Weak Points" list.
3. Create the `week_03.md` file. The first task is to address your weak points.
4. Outline the new topics for Week 3 in Year1.md

The plan is updated. The mission continues. Execute.

3.3 Week 3: Protocol - Integer Arithmetic and The Complete Solution

Source Path: year_01/quarter_01/planning/week3.tex

This week, we move from how numbers are stored to how they are manipulated. You will implement the logic of integer arithmetic at the bit level. In parallel, you will master the complete algorithm for solving any system of linear equations, the computational core of countless scientific problems. The work requires precision.

Monday: Computer Science - The Logic of Addition

- **Objective:** Understand addition, subtraction, and negation in complement a dos, including the critical concept of desbordamiento (overflow).
- **Block (2 Hours):** Read CS:APP, pp. 79-88 (Sección 2.3 a 2.3.3).
- **Execution:**

1. Realiza a mano la suma de 5 + 3 y 5 + 5 en complemento a dos de 4 bits, mostrando si ocurre desbordamiento y por qué.
2. Realiza a mano la resta 5 - 3 (calculada como 5 + (-3)) en complemento a dos de 4 bits.
3. Demuestra que negar el número más negativo en complemento a dos (ej. -8 en 4 bits) resulta en el mismo número, y explica la implicación de este caso límite.

Tuesday: Mathematics - Solving Ax = b

- **Objective:** Master the complete algorithm for solving systems of linear equations, including cases with no solution or infinite solutions.
- **Block (2-3 Hours):** Watch MIT 18.06, Lecture 7 ("Solving Ax = 0: Pivot Variables, Special Solutions") and Lecture 8 ("Solving Ax = b: Row Reduced Form R").
- **Execution:**

1. Toma una matriz 3x4 `A` y encuentra la solución completa para `Ax = 0`. Esto implica encontrar las variables pivote y libres, y construir las soluciones especiales que forman el nullspace.
2. Usando la misma matriz `A` y un vector `b`, encuentra la solución completa para `Ax = b`.
3. Explica con tus propias palabras la relación entre la solución particular (una solución a Ax=b) y el nullspace (todas las soluciones a Ax=0).

Wednesday: Computer Science - Multiplication by Machine

- **Objective:** Entender la multiplicación y la división por potencias de dos usando desplazamientos de bits, una optimización fundamental.
- **Block (2 Hours):** Read CS:APP, pp. 88-99 (Sección 2.3.4 a 2.3.8).
- **Execution:**

1. Multiplica 6 * 3 en binario sin signo de 4 bits, mostrando el resultado completo de 8 bits.

2. Escribe una función en C que multiplique un entero por 17 usando solo desplazamientos (`<<`) y sumas (`+`).

3. Explica cómo la división de un número negativo por una potencia de dos usando desplazamiento aritmético a la derecha (`>>`) difiere de la división matemática tradicional y cómo se puede corregir este sesgo.

Thursday: Mathematics - Independence, Basis, and Dimension

- **Objective:** Solidify your understanding of linear independence, spanning, basis, and dimension—the core concepts describing the "size" y la estructura de un espacio vectorial.
- **Block (2-3 Hours):** Watch MIT 1.06, Lecture 9 ("Linear Independence, Basis, and Dimension"). Dedica la sesión a resolver problemas del problem set asociado.
- **Execution:**

1. Dado un conjunto de 3 vectores en R^3, determina si son linealmente independientes.

2. Encuentra una base para el column space y una base para el nullspace de una matriz 3x4.

3. Para una matriz `m x n` de rango `r`, escribe las dimensiones de sus cuatro subespacios fundamentales (column space, nullspace, row space, left nullspace).

Friday: The Forge - Structuring Data

- **Objective:** Learn how to create complex data types using structs in Rust. This is the foundation for building any non-trivial application.
- **Block (2-3 Hours):** Read and work through Chapter 5 of *The Rust Programming Language* ("Using Structs to Structure Related Data").
- **Execution:**

1. Define a `User` struct with fields like `username`, `email`, y `active`.

2. Write functions that toman una instancia del `User` struct como argumento.

3. Implementa métodos en el `User` struct usando un bloque `impl`. Crea un método que devuelva la información del usuario como un string formateado.

Saturday: The Forge - Structs in Action

- **Objective:** Apply your knowledge of structs to a practical problem and explore more advanced features like tuple structs and debug printing.
- **Block (3-4 Hours):** Continue practicing concepts from Chapter 5 of the Rust book.
- **Execution:**

1. Crea un programa que calcule el área de un rectángulo, primero usando variables separadas, y luego refactorízalo para usar un `Rectangle` struct.

2. Refactoriza el programa de nuevo para usar métodos en el `Rectangle` struct, como `area()` y `can_hold()`.

3. Añade la anotación `#[derive(Debug)]` a tus structs y experimenta imprimiéndolos usando los especificadores de formato `{:?}` y `{:#?}`.

Sunday: Review and Reload

- **Objective:** Consolidate the week's knowledge, identify areas of weakness, and prepare the next week's operational plan.
- **Block (2 Hours):**
- **Execution:**

1. Review all notes. Focus on integer overflow, the process of finding the complete solution to `Ax = b`, y la sintaxis de los métodos `impl` en Rust.
2. Update your "Weak Points" list.
3. Create the `week_04.md` file. The first task is to address your weak points.
4. Outline the new topics for Week 4 in Year1.md

The pace is deliberate. The standard is absolute. Execute.

Source Path: year_01/quarter_01/planning/README.tex

3.3.1 CS:APP 2nd Edition - Assault Protocol

This book is the doctrinal backbone for Year 1 systems knowledge. It will be approached methodically, one chapter per month, to ensure deep mastery over superficial understanding.

Preliminary Mission: Chapter 1 - Reconnaissance

Chapter 1, "A Tour of Computer Systems," is a high-level operational overview. **Read it once, quickly, for context.** Do not get bogged down in details. Its purpose is to give you a map of the territory you will explore in depth in the following chapters. You have already covered it in Week 1. Refer back to it only if you need to recall how the pieces fit together.

Month 1: Chapter 2 - Representing Information

This is the true starting point. The objective is to master how data exists at the bit level. The chapter is divided into a four-week assault.

- **Week 1: Foundations of Representation**
- **Monday:** Read pp. 29-46 (Section 2.1.1 to 2.1.4).
- **Focus:** Hexadecimal notation, `word size`, and byte ordering (endianness).
- **Wednesday:** Read pp. 46-56 (Section 2.1.5 to 2.1.10).
- **Focus:** String representation, bit-level operations (`&`, `|`, `^`, `~`), and shift operations (`<<`, `>>`).
- **Week 2: Integer Representation**
- **Monday:** Read pp. 56-69 (Section 2.2 to 2.2.4).
- **Focus:** Unsigned integers and the logic of two's complement for signed integers.
- **Wednesday:** Read pp. 69-79 (Section 2.2.5 to 2.2.8).
- **Focus:** Conversions between signed/unsigned, sign extension, and the dangers of truncation.
- **Week 3: Integer Arithmetic**
- **Monday:** Read pp. 79-88 (Section 2.3 to 2.3.3).
- **Focus:** Two's complement addition, subtraction, and negation; overflow detection.
- **Wednesday:** Read pp. 88-99 (Section 2.3.4 to 2.3.8).
- **Focus:** Multiplication and the optimization of division by powers of two via shifts.
- **Week 4: Floating-Point Arithmetic**
- **Monday:** Read pp. 99-110 (Section 2.4 to 2.4.3).
- **Focus:** Fractional binary representation and the IEEE 754 standard (sign, exponent, fraction).

- **Wednesday:** Read pp. 110-118 (Section 2.4.4 to 2.5).
- **Focus:** Rounding modes, floating-point operations, and the nature of precision errors.

- Homework problems (pp. 119 - 134)
- Solutions (pp. 134 - 151)

Month 2: Chapter 3 - Machine-Level Program Representation

This month, you move from *what* data is to *how* the machine acts upon it. The objective is to learn to read and understand x86-64 assembly language. This is not an academic exercise; it is the process of learning the machine's native tongue. The pace is aggressive. It requires absolute focus.

- **Week 5: Assembly Fundamentals**
- **Monday:** Read pp. 156-177 (Sections 3.1 to 3.4).
- **Focus:** Program encodings, data formats, accessing information, and the roles of the primary integer registers.
- **Wednesday:** Read pp. 177-199 (Sections 3.5 to 3.6).
- **Focus:** Arithmetic and logical operations (`leaq`, `add`, shifts) and the fundamentals of control flow (condition codes, jumps).
- **Week 6: Procedures and Arrays**
- **Monday:** Read pp. 200-218 (Section 3.7).
- **Focus:** The run-time stack, control transfer (`call`/`ret`), and register usage conventions for procedures.
- **Wednesday:** Read pp. 219-240 (Section 3.8).
- **Focus:** Array allocation, pointer arithmetic, and the layout of nested and variable-sized arrays in memory.
- **Week 7: Data Structures and Pointers**
- **Monday:** Read pp. 241-253 (Section 3.9 to 3.10).
- **Focus:** The memory layout of `structs` and `unions`, data alignment, and a consolidated understanding of pointers.
- **Wednesday:** Read pp. 254-266 (Sections 3.11 to 3.12).
- **Focus:** Practical use of the GDB debugger and the critical security implications of buffer overflow vulnerabilities.
- **Week 8: x86-64 and Floating Point**
- **Monday:** Read pp. 267-280 (Section 3.13).
- **Focus:** The extension to 64-bit, how information is accessed, and control flow in x86-64.
- **Wednesday:** Read pp. 280-294 (Sections 3.13 to 3.15).

- **Focus:** Data structures in x86-64, machine-level representation of floating-point programs, and the chapter summary.

- **Homework Problems:** (pp. 294 - 308)
- **Solutions to Practice Problems:** (pp. 308 - ...)